can also use `miner.start(6)`, in which six parallel threads are assigned to the mining operation. By the way, *mining* is the process of selection a new block for the chain.

You can explore many more packages based on the problem you are trying to solve. You can always install web3 (`require("web3")`) in your Node.js environment, connect to the web3 provider in the Ganache test chain, and test the web3 commands from the command line to learn how they operate before coding them into your application.

7.5 Retrospective

You got firsthand experience with digital signing in this chapter. Even though this feature was initiated in the application part (MPC-app) of the Dap, it used the blockchain services for hashing and signing the message. Web3 accessed the hashing and signing functions of the web3 provider in the Ganache node to accomplish these tasks (hashing and signing). It is very important to note that you cannot use any hashing method you wish and sign with anything you like. You are in a common blockchain network; everyone has to speak the same language and follow methods offered by the blockchain services. That’s what you did. You used the web3-provided SHA3 function for hashing and used the private key of the account for signing with the help of web3 and MetaMask.

The smart contract for the micropayment channel was quite simple, with a constructor and two public functions to validate the signature and to pay the claim. The other functions in the smart contract accessed the underlying blockchain functions to get you the signer of the micropayment message. Once again, you must use the standard functions to accomplish the tasks in a